

UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE MATEMÁTICA, ESTATÍSTICA E COMPUTAÇÃO
CIENTÍFICA

RELATÓRIO - MS428

**Implementação do Algoritmo Simplex Primal para Resolução
de Problemas Lineares de Ordem Média**

Aluno:
João Vitor Hartung Toppa
213140

Aluno:
Laura Torquato Rangel

Campinas, SP
18 de outubro de 2025

18 de outubro de 2025

Sumário

1	Introdução	2
1.1	Conceitos	2
2	Objetivo	5
3	Metodologia	6
3.1	Implementação do Algoritmo Simplex	6
3.1.1	Estrutura do Algoritmo Simplex	6
3.1.2	Funções do algoritmo Simplex	7
3.2	Codificação dos Problemas Lineares	9
4	Resultados	11
4.1	Testes de Factibilidade	11
4.1.1	Problema Factível	11
4.1.2	Problema Infactível	12
4.1.3	Problema Ilimitado	13
4.2	Análise do Algoritmo Implementado	15
4.2.1	Tempo de Execução $\times$ Uma Iteração	15
4.2.2	Tempo de Execução para PLs de Tamanho Fixo	15
4.2.3	Média de Tempo de Execução $\times$ Resolução de PLs	17
5	Discussão	19
6	Conclusão	20
A	Prints - Testes de factibilidade	22
B	Código em Python para Geração de PLs	26

1 Introdução

A determinação de soluções ótimas para sistemas de desigualdades lineares assume crescente importância como ferramenta para a análise matemática de certos problemas como os problemas de fluxo *multicommodity* ou os problemas de *mix de produção*.

O algoritmo ou método Simplex de Dantzig é um algoritmo para resolução de problemas do ramo da pesquisa operacional que propõe uma maneira de descobrir a solução de um problema de otimização linear em tempo exponencial [1]. O método baseia-se no fato de que, geometricamente, restrições lineares definem um politopo convexo, chamado de conjunto de pontos viáveis. A propriedade de convexidade pertinente à região factível garante que todo mínimo local é um mínimo global - e, além disso, que o mínimo global pertence necessariamente a pelo menos um dos vértices do politopo. Esse vértice é chamado de solução básica factível (BFS). O método simplex utiliza essa propriedade para avaliar a qualidade dos vértices como candidatos a solução ótima.

A solução de um programa linear é realizada em duas etapas. Na primeira etapa, conhecida como Fase I, um ponto extremo inicial é encontrado. Dependendo da natureza do programa, isso pode ser trivial, mas em geral pode ser resolvido aplicando o algoritmo Simplex a uma versão modificada do programa original. Os resultados possíveis da Fase I são que uma solução básica factível é encontrada ou que a região factível é vazia. Neste último caso, o programa linear é chamado de inviável ou infactível. Na segunda etapa, Fase II, o algoritmo simplex é aplicado usando a solução básica viável encontrada na Fase I como ponto de partida. Os resultados possíveis da Fase II são uma solução básica viável ótima ou uma aresta infinita na qual a função objetivo é ilimitada [2]. O procedimento seguido pelo algoritmo ocorre de maneira bastante análoga à solução de equações lineares ou inversão de matrizes por eliminação gaussiana. Como esta última, é útil na demonstração de teoremas fundamentais sobre sistemas algébricos lineares [3].

Dado a importância do método, o presente projeto tem como objetivo demonstrar uma implementação do método Simplex em linguagem de programação C++. Espera-se que o algoritmo implementado seja capaz de solucionar problemas de pequeno (matrizes de restrições $< 500 \times 500$) e médio ($< 1000 \times 1000$) porte, sendo a resolução limitada somente pela capacidade de processamento e de armazenamento em RAM do computador utilizado.

1.1 Conceitos

Simplex Primal O método Simplex é o algoritmo dominante em uso para resolver PLs. Ele apresenta complexidade de tempo exponencial no pior caso, mas seu desempenho na prática tem sido excelente e está sendo continuamente aprimorado pelo desenvolvimento de tecnologias de implementação. Existem muitas variantes do método Simplex, a mais proeminente sendo o método Simplex primal.

Este método exige um vetor básico inicial viável. Se um vetor básico viável primal não estiver disponível, o método introduz variáveis artificiais no problema e configura um problema de Fase I com um vetor básico viável prontamente disponível, consistindo de variáveis básicas artificiais. Quando este problema de Fase I é resolvido pelo mesmo algoritmo, ao término ele fornece um vetor básico viável para o primal original ou uma prova de que ele é inviável [4].

Forma Padrão Define-se a forma padrão de um problema linear como:

$$\min c^T x$$

$$\begin{cases} Ax = b \\ x \geq 0 \end{cases}$$

Onde c e x são vetores com n componentes (quantidade de variáveis), b é um vetor com m componentes (quantidade de restrições), e A é uma matriz de dimensões $m \times n$.

Para resolver este problema, dividimos a matriz A em duas partes: uma matriz A_1 de dimensão $m \times (n - j)$ e uma matriz identidade I' de dimensão $m \times j$, onde j é o número de colunas da identidade I_m , com m linhas. Montamos então uma nova matriz A' de dimensão $m \times (n + m - j)$, e atualizamos os vetores x e c , que se tornam x' e c' , com $n + m - j$ componentes, na fase 1.

Na fase 2, a matriz A' é dividida em duas partes: B , uma matriz quadrada $m \times m$, e N , uma matriz de dimensões $m \times (n - j)$. Além disso, os vetores x' e c' são divididos em x_b , c_b , x_n e c_n , onde x_b e c_b têm m componentes e correspondem às variáveis básicas, enquanto x_n e c_n têm $n - j$ componentes e correspondem às variáveis não-básicas.

Definimos também o vetor multiplicador simplex λ , que tem o mesmo tamanho de c_b , e os custos relativos c_{n_i} para cada variável x_{n_i} . O vetor y é o vetor da direção simplex.

O algoritmo Simplex busca particionar a matriz A em duas partes:

1. A *matriz básica*, que corresponde ao vértice avaliado;
2. A *matriz não-básica*, que corresponde aos possíveis candidatos a vértices.

O método, portanto, consiste em atualizar a base de modo que esta base contenha as variáveis que minimizem a função objetivo.

Big M Para completar a partição básica, considerando que a identidade I_m é de tamanho $m \times m$, podemos evitar depender diretamente da matriz A para iniciar o método. Para isso, ao considerarmos a matriz aumentada A'' , devemos adicioná-la de forma a incorporar $m - k + 1$ linhas e colunas, de maneira que a matriz A seja atualizada para incluir as colunas faltantes da identidade.

O objetivo dessa atualização é adicionar $m - k$ colunas de forma que a matriz A' contenha uma parte da identidade I_m , com o restante dos elementos vindo de A'' .

Para que isso funcione corretamente, não apenas a matriz A precisa ser atualizada, mas também o vetor de variáveis. Ao adicionar $m - j$ novas variáveis à base, precisamos garantir que essas variáveis não influenciem a solução final do problema. Como o problema é formulado na forma padrão (e, portanto, é um problema de minimização), atribuímos um custo muito alto para essas novas variáveis.

Uma vez feitas essas atualizações, o problema está pronto para ser resolvido com o método Simplex. Durante a primeira iteração, a partição básica será dada pela identidade, ou seja, $B = I$, e a partição não-básica será composta pelas variáveis de A'' . Como resultado, a primeira iteração terá valores positivos, o que assegura que sempre teremos uma Partição Inicial Factível.

Além disso, como estamos garantindo que as variáveis adicionadas (com custos muito altos) não farão parte da solução final, elas sairão da base nas primeiras m iterações do algoritmo. Esse processo assegura que, se o problema for factível, a base será atualizada de forma adequada, buscando a solução ótima e minimizando a função objetivo [1].

Eliminação Gaussiana Para resolver um problema de programação linear no método simplex, precisamos resolver alguns sistemas lineares:

1. Durante o encontro de soluções básicas;
2. Durante o cálculo de λ ;
3. Durante o cálculo de y .

O algoritmo aplicado para solução de sistemas lineares - eliminação gaussiana - é uma eliminação sequencial das variáveis em cada equação, até que cada equação tenha apenas uma variável

restante. Se $n = m$, pode-se pensar nele como a transformação da matriz A na matriz identidade. O algoritmo para eliminação por Gauss-Jordan está representado abaixo [5]:

Algorithm 1 Método de Eliminação de Gauss-Jordan

```

 $h := 1$  ▷ Inicialização da linha pivô
 $k := 1$  ▷ Inicialização da coluna pivô
while  $h \leq m$  e  $k \leq n$  do
  Encontrar o  $k$ -ésimo pivô:
   $i_{\max} := \arg \max_{i=h \dots m} |A[i, k]|$ 
  if  $A[i_{\max}, k] = 0$  then
    Não há pivô nesta coluna, passa para a próxima coluna
     $k := k + 1$ 
  else
    Trocar as linhas  $h$  e  $i_{\max}$ 
    for  $i = h + 1 \dots m$  do
       $f := \frac{A[i, k]}{A[h, k]}$ 
      Preencher com zeros a parte inferior da coluna pivô:  $A[i, k] := 0$ 
      for  $j = k + 1 \dots n$  do
         $A[i, j] := A[i, j] - A[h, j] \cdot f$ 
      end for
    end for
    Aumentar a linha e a coluna pivô:  $h := h + 1, k := k + 1$ 
  end if
end while

```

A eliminação gaussiana é baseada em duas relações simples:

1. É sempre possível trocar duas equações (linhas).
2. Qualquer equação pode ser substituída por uma combinação linear dessa linha (com coeficiente não-zero) e de outras linhas (com coeficientes arbitrários).

Na primeira etapa, o algoritmo de Gauss-Jordan divide a primeira linha por a_{11} . Em seguida, o algoritmo adiciona a primeira linha às linhas restantes de modo que os coeficientes na primeira coluna se tornem todos zeros. Para isso, na i -ésima linha, devemos adicionar a primeira linha multiplicada por $-a_{i1}$. Observe que essa operação também deve ser realizada no vetor b . De certa forma, ela age como se o vetor b fosse a $(m + 1)$ -ésima coluna da matriz A .

Como resultado, após a primeira etapa, a primeira coluna da matriz A terá 1 na primeira linha e 0 nas outras linhas.

De maneira similar, realizamos a segunda etapa do algoritmo, onde consideramos a segunda coluna da segunda linha. Primeiro, a linha é dividida por a_{22} , depois é subtraída das outras linhas de modo que toda a segunda coluna se torne 0 (exceto pela segunda linha). Estendemos essas operações para todas as linhas e colunas, para então encontrarmos a resolução do sistema linear [6].

Formato de Arquivo DIMACS O formato DIMACS é um formato padrão utilizado para representar grafos não direcionados em arquivos de entrada. O formato de arquivo é composto por diferentes tipos de linhas, conforme descrito abaixo:

- **Comentários:** Linhas de comentário fornecem informações legíveis por humanos e são ignoradas pelos programas. As linhas de comentário começam com a letra minúscula “c” e podem aparecer em qualquer lugar no arquivo. Exemplos de linha de comentário:

c Esta é uma linha de comentário.

- Linha do Problema: Há exatamente uma linha de problema por arquivo de entrada. Ela deve aparecer antes de qualquer linha de descritores de nós ou arestas e tem o seguinte formato:

p FORMAT , NODES , EDGES

Onde:

- FORMAT: o formato do arquivo, que deve ser a palavra "edge".
- NODES: um valor inteiro n representando o número de nós no grafo.
- EDGES: um valor inteiro m representando o número de arestas no grafo.

- Descritores de Arestas: Para cada aresta no grafo, existe uma linha de descritor de aresta com o seguinte formato:

e u v

Onde:

- e: a letra minúscula que indica que esta linha descreve uma aresta.
- u: o ponto final u da aresta.
- v: o ponto final v da aresta.

Cada aresta (u, v) aparece exatamente uma vez no arquivo de entrada [8].

2 Objetivo

O objetivo principal deste trabalho é a implementação do algoritmo Simplex para a resolução de problemas de pequeno ($< 500 \times 500$) e médio ($< 1000 \times 1000$) porte. Objetivos secundários envolvem:

1. Um gerador de problemas lineares na forma padrão, para testes e geração de um conjunto de problemas-teste.
2. Um sistema de armazenamento serializado de problemas lineares, para construir o conjunto de problemas-teste de uma forma facilmente armazenável e comentável.

Busca-se disponibilizar, também, todos os códigos desenvolvidos para a comunidade como forma de validação dos resultados obtidos por este projeto.

3 Metodologia

A implementação do algoritmo sugerido neste relatório seguiu as etapas típicas da engenharia de algoritmos (projeto, análise, implementação e experimentos) [7] em ciclos de refinamento sucessivo e incremento de funcionalidades. Além disso, percebeu-se a necessidade de avaliar os métodos implementados por meio de um conjunto de testes. Para isso, portanto, implementamos tanto o algoritmo Simplex quanto um método de representação de PLs na forma padrão para facilitar testes de entrada. Tanto o método desenvolvido quanto o método de criação do conjunto de testes estão descritos abaixo.

3.1 Implementação do Algoritmo Simplex

3.1.1 Estrutura do Algoritmo Simplex

A versão do algoritmo implementada realiza duas fases: a Fase 1, responsável por definir os vetores de índices de colunas básicas e não-básicas; a Fase 2, responsável por calcular os custos relativos, o vetor multiplicador simplex, a direção simplex e atualizar os vetores de índices. Abaixo, apresentamos um fluxograma para o algoritmo simplex.

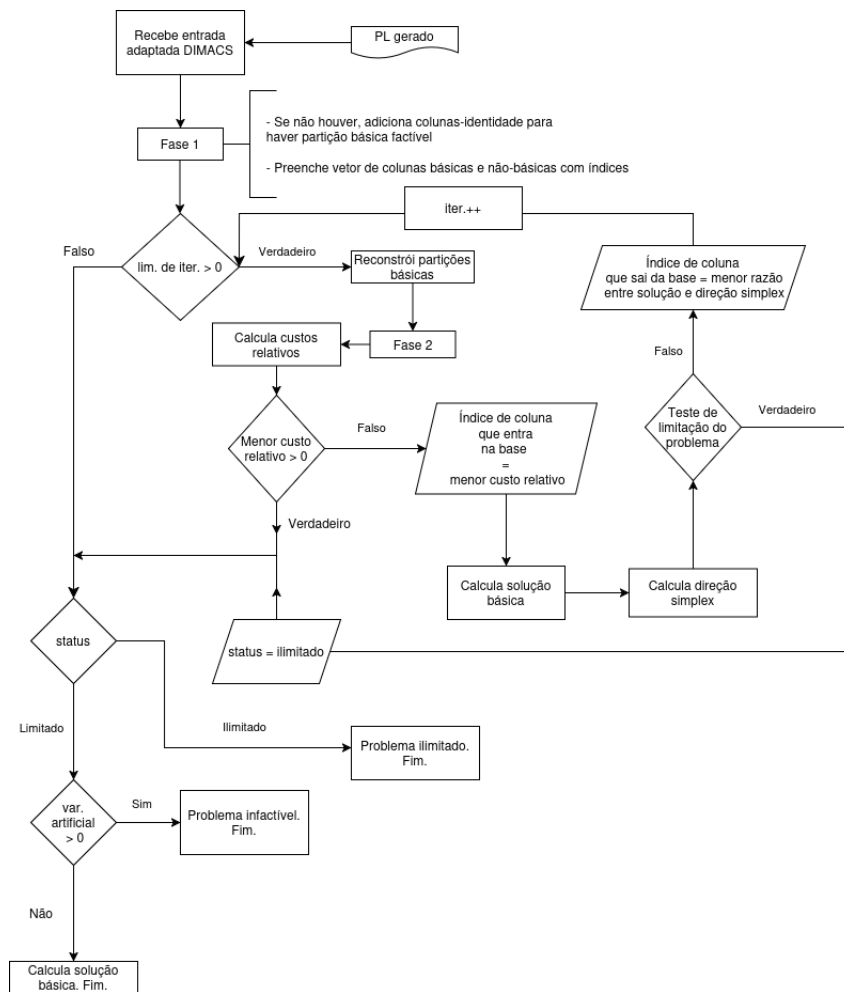


Figura 1: Fluxograma para o algoritmo Simplex. Elaborado pelos autores (2025).

3.1.2 Funções do algoritmo Simplex

Abaixo, descrevemos independentemente detalhes de implementação de cada função do algoritmo.

Fase 1 Na solução de um problema de PL na forma padrão, é necessário definir as partições básica e não básica. A partição básica é composta por m variáveis, correspondendo a uma matriz $m \times m$, enquanto as variáveis não básicas correspondem às colunas restantes da matriz A .

O objetivo da Fase 1 é identificar uma solução inicial factível, de modo que todas as variáveis sejam não-negativas. Considerando a forma padrão, o vetor de recursos b é não-negativo, e a solução inicial factível pode ser obtida ao garantir que a partição básica seja composta pelas colunas da matriz identidade.

Para o algoritmo, a partição inicial básica será dada pela matriz identidade I_m , ou seja, as colunas da partição básica serão as colunas da identidade já presentes mais colunas Big M suficientes para que a matriz I_m seja $m \times m$. A partir disso, o vetor x_b (solução básica) será igual ao vetor de recursos b , uma vez que $x_b = b$ e x_b é não-negativo.

O algoritmo identifica as colunas de A que correspondem a colunas da identidade. Para isso, ele busca por colunas em A que possuam um único valor igual a 1, com os demais elementos da coluna iguais a 0. Essas colunas são as que formam a matriz identidade, desde que as novas colunas identificadas não tenham sido utilizadas anteriormente. Após a identificação dessas colunas, a matriz A é então dividida conforme segue:

$$A = (A' \mid I')$$

Onde:

- A' representa as colunas que não pertencem à matriz identidade.
- I' representa as colunas que formam a matriz identidade.

Em resumo, se na matriz A'' já encontramos k colunas da identidade, precisamos adicionar $m - k$ colunas para completar a matriz A' , transformando-a em uma matriz que contém a identidade I_m . Ou seja, chamando os elementos de A'' de a_{ij} , os de I' de i'_{ij} e os da identidade acoplada de i_{ij} , podemos reorganizar as colunas da seguinte forma:

$$\begin{pmatrix} a_{11} & \cdots & a_{1n-k} & i'_{11} & \cdots & i'_{1k} & i_{1k+1} & \cdots & i_{1m} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn-k} & i'_{m1} & \cdots & i'_{mk} & i_{mk+1} & \cdots & i_{mm} \end{pmatrix} = \begin{pmatrix} a_{11} & \cdots & a_{1n-k} & 1 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn-k} & 0 & \cdots & 1 \end{pmatrix}$$

Fase 2 Com a Fase 1 pronta, temos uma partição básica para começar o método e conseguimos obter o vetor de índices de colunas básicas B , as variáveis das colunas que montam a matrix B (x_b), o vetor de índices de colunas não-básicas N e as variáveis das colunas que montam a matrix N (x_n). Assim, conseguimos começar o método. A fase 2 é dividida em quatro partes:

1. Cálculo dos custos relativos;
2. Cálculo da solução básica;
3. Cálculo da direção simplex;
4. Atualização das dos vetores de índices básicos/não-básicos.

Para fins de implementação, Fase 2 é uma função *wrapper* para as quatro funções descritas acima. Um detalhamento maior sobre cada uma das funções é realizado abaixo, assim como uma visão geral em formato de pseudocódigo do fluxo de utilização das funções.

Algorithm 2 Fase 2

```

procedure PHASE2(basicPartition, basicColumns, costVector, restrictionsM, nonBasicPartition,
nonBasicColumns, resources)
     $rc \leftarrow \text{RELATIVECOSTS}(\text{basicPartition}, \text{basicColumns}, \text{costVector}, \text{nonBasicPartition}, \text{nonBasicColumns})$ 
    if  $rc.\text{lowestRC} \geq -\epsilon$  then                                /* Esta condição também evita ciclagens! */
        return -1
    end if
     $x_B \leftarrow \text{BASICSOLUTION}(\text{basicPartition}, \text{basicColumns}, \text{costVector}, \text{resources})$ 
     $\text{enteringCol} \leftarrow rc.\text{lowestNonBColumn}$ 
     $k \leftarrow -1$ 
    for  $i = 0$  to  $\text{nonBasicColumns.size}() - 1$  do
        if  $\text{nonBasicColumns}[i] = \text{enteringCol}$  then
             $k \leftarrow i$ 
            break
        end if
    end for
     $y \leftarrow \text{SIMPLEXDIRECTION}(\text{basicPartition}, \text{nonBasicColumns}, \text{nonBasicPartition}, k)$ 
    foundPositive  $\leftarrow$  false
    for  $i = 0$  to  $y.\text{size}() - 1$  do
        if  $y[i] > \epsilon$  then
            foundPositive  $\leftarrow$  true
            break
        end if
    end for
    if not foundPositive then
        return -2
    end if
     $\text{leavingColumn} \leftarrow \text{OUTBASIS}(y, x_B, \text{basicColumns})$ 
     $\text{UPDATEBASIS}(\text{basicColumns}, \text{nonBasicColumns}, \text{enteringCol}, \text{leavingColumn})$ 
    return 0
end procedure

```

Custos relativos Para chegar na otimalidade precisamos verificar o quanto cada uma das variáveis impactam na função objetivo, ou seja, o quanto proporcionalmente a mudança da base pode ativamente modificar a função objetivo. Seja o vetor $x_n = (x_{n_1}, \dots, x_{n_{n-j}})$, matriz $N = (a_{n_1}, \dots, a_{n_{n-j}})$, onde a_{n_i} são colunas da matriz, e $c_n = (c_{n_1}, \dots, c_{n_{n-j}})$, resolvemos a equação:

$$c'_{n_i} = c_{n_i} - \lambda a_{n_i}$$

Para cada $i = (1, \dots, n - j)$. Definimos λ como o Vetor Multiplicador Simplex (VMS), representando o custo das mudanças nas variáveis básicas necessárias para satisfazer as restrições $Ax = b$. Para encontrar λ , resolvemos, usando eliminação gaussiana, o sistema linear:

$$B^t \lambda = c_b$$

Com isso, obtemos o quanto cada variável a entrar na base impacta a função objetivo por unidade. Como estamos buscando resolver um problema de minimização, buscamos o mínimo entre esses valores e checamos se ele é negativo. Chamamos essa variável de x_{n_k} . Dessa forma, temos que se colocarmos

x_{n_k} em x_b , a função objetivo irá diminuir. Se x_{n_k} for não negativo, então temos que a solução é ótima, pois toda variável que não está na base tem apenas o potencial de aumentar a função objetivo. Se obtivemos custos relativos não negativos, então obtivemos o vértice ótimo.

Solução Básica A solução básica é obtida resolvendo o sistema linear $B \cdot x_B = b$, onde B é a submatriz formada pelas colunas básicas da matriz de restrições A , e x_B é o vetor das variáveis básicas.

Para encontrar essa solução, primeiramente a matriz B é ampliada com o vetor b , formando uma matriz aumentada $[B \mid b]$. Essa matriz é então resolvida utilizando o método de eliminação de Gauss, que calcula o vetor x_B . Após a resolução do sistema, qualquer valor em x_B que seja menor que um valor de tolerância (definido como $\epsilon = 10^{-7}$) é ajustado para zero, de modo a eliminar possíveis erros numéricos causados por arredondamentos.

Com a solução básica x_B calculada, a função objetivo também é avaliada. Para isso, as variáveis básicas são multiplicadas pelos respectivos custos e a soma desses produtos fornece o valor da função objetivo para a solução atual. A função então retorna o vetor x_B , que contém os valores das variáveis básicas que satisfazem o sistema de restrições do problema.

Direção Simplex Caso x_{n_k} seja negativo, precisamos escolher uma variável para sair da base. Para isso, calculamos a direção simplex y resolvendo o seguinte sistema linear:

$$By = a_{n_k}$$

Dessarte, checando se o vetor y possui valores estritamente positivos vemos as possibilidades de variáveis que possam sair da base. Se o vetor não possuir valores positivos, isso indica que o problema é ilimitado, e o programa para. Agora, apenas para os valores positivos de y , escolhemos o melhor valor para sair da base tomando x_{b_l} para qual a divisão $\frac{x_{b_l}}{y_l}$ seja mínima. Este menor valor equivale à variável que sairá da base.

Atualização das Bases Com os passos anteriores, temos a direção a ser percorrida definida. Para alcançar a otimalidade, ou apenas chegar mais perto dela, vamos remontar as matrizes B e N , da forma que x_{n_k} entre no vetor de índices básicos e x_{b_l} entre no vetor de índices não-básico. Para isso, reconstruímos as partições atualizadas e começamos a fase 2 de novo até chegar em uma solução em que os custos relativos das variáveis não-básicas sejam não-negativos.

3.2 Codificação dos Problemas Lineares

Em um PL na forma padrão, necessitamos dos seguintes elementos como entrada:

- m : Número de restrições (linhas) no problema.
- n : Número de variáveis (colunas) no problema.
- c : Vetor dos custos, representado por $\mathbf{c} \in \mathbb{R}^n$, onde cada entrada c_i é o custo associado à variável x_i .
- b : Vetor dos recursos (ou termos independentes), representado por $\mathbf{b} \in \mathbb{R}^m$, onde cada entrada b_i é o valor do recurso na restrição i .
- A : Matriz dos coeficientes das restrições, representada por $A \in \mathbb{R}^{m \times n}$, onde cada entrada a_{ij} representa o coeficiente da variável x_j na restrição i .

Para tanto, necessitamos desenvolver algum tipo de metodologia para permitir entrada e saída de forma automatizada. A solução proposta utiliza uma variação do formato DIMACS [8] para

entradas. O método `receiveInputFile` tem como objetivo ler e processar um arquivo de entrada contendo as informações necessárias para definir um PL. Abaixo, detalhamos como esse processo ocorre.

Parâmetros de Entrada O método recebe os seguintes parâmetros:

1. **filename**: Caminho para o arquivo de entrada.
2. **costVector**: Vetor que armazenará os coeficientes da função objetivo.
3. **A**: Matriz de coeficientes das restrições.
4. **resources**: Vetor que armazenará os termos independentes das restrições.
5. **constraints**: Número de restrições (linhas da matriz *A*).
6. **variables**: Número de variáveis (colunas da matriz *A*).

O arquivo é aberto utilizando um fluxo de entrada (`ifstream`). Se o arquivo não puder ser aberto, o método imprime uma mensagem de erro e retorna `-1`. Então, lê linha a linha utilizando a função `getline`. Cada linha é analisada para identificar seu tipo e extrair os dados correspondentes:

1. Linhas iniciadas com o caractere `#` são ignoradas.
2. Quando a linha começa com o caractere `p`, ela contém as informações sobre o número de variáveis e restrições. Esses valores são extraídos e armazenados nas variáveis **variables** e **constraints**.
3. As linhas que começam com `a` contêm os coeficientes das restrições. Esses coeficientes são extraídos e armazenados em um vetor unidimensional chamado **a_flat**.
4. As linhas que começam com `b` contêm os valores dos recursos. Esses valores são extraídos e armazenados no vetor **resources**.
5. As linhas que começam com `c` contêm os coeficientes da função objetivo. Esses valores são extraídos e armazenados no vetor **costVector**.

Após a leitura das linhas, o algoritmo verifica se o número de coeficientes das restrições lidos é compatível com as dimensões especificadas na linha do problema. Ou seja, o número total de elementos no vetor **a_flat** deve ser igual ao produto entre **constraints** e **variables**. Se houver discrepância, o método retorna `-1`. O vetor unidimensional **a_flat** é reorganizado em uma matriz bidimensional *A* com o número adequado de linhas e colunas, conforme as variáveis **constraints** e **variables**. Após processar o arquivo, o fluxo de entrada é fechado. Ao final da execução, os dados lidos são armazenados nas variáveis e estruturas passadas como parâmetros:

- **costVector** contém os coeficientes da função objetivo.
- **A** contém a matriz de coeficientes das restrições.
- **resources** contém os termos independentes das restrições.
- **constraints** e **variables** contêm o número de restrições e variáveis, respectivamente.

Caso o processo de leitura seja bem-sucedido, o método retorna `0`, indicando que todos os dados foram processados corretamente. Caso contrário, retorna `-1`. Suponha que o arquivo de entrada siga o formato abaixo:

```

p 3 2
c 1 2
a 1 1
a 2 3
b 5 8

```

Neste exemplo, o arquivo especifica um problema de Programação Linear com 3 variáveis, 2 restrições, a função objetivo com coeficientes [1, 2], coeficientes das restrições [1, 1] e [2, 3], e os recursos [5, 8]. De modo a validar o algoritmo desenvolvido, um código em Python para gerar PLs com este formato adaptado foi desenvolvido. Este pode ser encontrado no Apêndice B.

É importante notar que este tipo de entrada é opcional. À caráter do usuário, pode-se utilizar tanto inserção manual quanto este modelo de entrada alternativo.

4 Resultados

Nesta seção, apresentamos os resultados obtidos de análise estatística e de testes preliminares do algoritmo desenvolvido. O algoritmo primal Simplex foi implementado em C++ seguindo as etapas típicas da engenharia de algoritmos [7] (projeto, análise, implementação e experimentos). O código implementado está disponível em <https://github.com/jvtoppa/LP-solver>. Para executar o script, basta digitar `make` em um terminal Linux ou compilar manualmente com o comando `g++ -std=c++11 -O3 -DNDEBUG -I$HOME/include simplex.cpp linearalgebra.cpp -o simplex`. É importante verificar que o computador disponível possui o compilador GNU C Compiler instalado previamente.

Dentre as ferramentas utilizadas, destacam-se a STL, biblioteca padrão de C++, e a biblioteca `chrono`. Os resultados obtidos por experimentação foram executados em um computador munido de processador Intel Core i5-13420H e 8GiB de memória RAM disponível.

4.1 Testes de Factibilidade

Abaixo, demonstramos o comportamento do algoritmo ao lidar com problemas factíveis, infactíveis e ilimitados. Mais especificamente, apresentamos o conteúdo impresso diretamente no terminal. *Prints* de tela do algoritmo estão disponíveis no apêndice.

4.1.1 Problema Factível

Com entrada:

```

6
3

10 2 1 1 0 0
7 3 2 0 1 0
2 4 1 0 0 1

-22 -6 -2 0 0 0

100 72 80

```

Obtemos o seguinte problema de minimização:
Minimizar:

$$z = -22x_0 - 6x_1 - 2x_2 + 0x_3 + 0x_4 + 0x_5$$

Sujeito a :

$$\begin{cases} 10x_0 + 2x_1 + x_2 + x_3 + 0x_4 + 0x_5 = 100 \\ 7x_0 + 3x_1 + 2x_2 + 0x_3 + x_4 + 0x_5 = 72 \\ 2x_0 + 4x_1 + x_2 + 0x_3 + 0x_4 + x_5 = 80 \end{cases}$$

Iteration 0

Enters Non basic column: 0

Lowest Relative Cost: -22

f(x) = 0

Basic column that leaves basis: 3

Iteration 1

Enters Non basic column: 1

Lowest Relative Cost: -1.6

f(x) = -220

Basic column that leaves basis: 4

Iteration 2

Enters Non basic column: 4

Lowest Relative Cost: 1

Exited. Optimal solution found.

f(x) = -222

Final solution:

f(x) = -222

x[0] = 9.75

x[1] = 1.25

x[2] = 0

x[3] = 0

x[4] = 0

x[5] = 55.5

Simplex Method - Time elapsed: [0ms]

4.1.2 Problema Infactível

Com entrada:

2

1 1

1 1

1 1

3 2

Obtemos o seguinte problema de minimização:

Minimizar:

$$x_0 + x_1$$

Sujeito a:

$$\begin{cases} x_0 + x_1 = 3 \\ x_0 + x_1 = 2 \end{cases}$$

Iteration 0

Enters Non basic column: 0

Lowest Relative Cost: -2e+09

f(x) = 5e+09

Basic column that leaves basis: 3

Iteration 1

Enters Non basic column: 1

Lowest Relative Cost: 0

Exited.

f(x) = 1e+09

INFEASIBLE: artificial variable x[2] = 1 > 1e-07

Simplex Method - Time elapsed: [0ms]

4.1.3 Problema Ilimitado

Com entrada:

5

2

1 0 -1 1 0

0 1 -1 0 1

-1 -1 -1 0 0

2 3

Obtemos o seguinte problema de minimização:

Minimizar:

$$-x_0 - x_1 - x_2 + 0x_3 + 0x_4$$

Sujeito a:

$$\begin{cases} x_0 + 0x_1 - x_2 + x_3 + 0x_4 = 2 \\ 0x_0 + x_1 - x_2 + 0x_3 + x_4 = 3 \end{cases}$$

Iteration 0

Enters Non basic column: 2

Lowest Relative Cost: -3

f(x) = -5

Problem is unbounded (no positive y).

The problem is unbounded.

Simplex Method - Time elapsed: [0ms]

4.2 Análise do Algoritmo Implementado

A fim de avaliar a eficácia do algoritmo desenvolvido, foram realizados experimentos com a geração de problemas de natureza aleatória. O objetivo principal foi analisar o desempenho do algoritmo, tanto em termos de precisão quanto de tempo de execução. Para tanto, foi implementado um código responsável pela geração automatizada de problemas "comportados", que foram então fornecidos na estrutura de entrada requerida. Após a resolução de cada instância, os resultados obtidos, juntamente com os tempos de execução, foram utilizados para gerar gráficos comparativos, permitindo uma análise detalhada da eficiência e qualidade do algoritmo Simplex implementado.

4.2.1 Tempo de Execução $\times$ Uma Iteração

De modo a validar a implementação, analisamos o impacto do tamanho de um PL, considerando o número de variáveis, sobre o tempo de resolução de uma iteração. Para isso, geramos diversas instâncias de PLs de matriz de restrições $N \times N$, variando N de 10 em 10, começando de 10 até 1500 variáveis. Em seguida, medimos o tempo de execução de uma iteração do algoritmo, buscando avaliar o tempo necessário para o código realizar o cálculo de uma base.

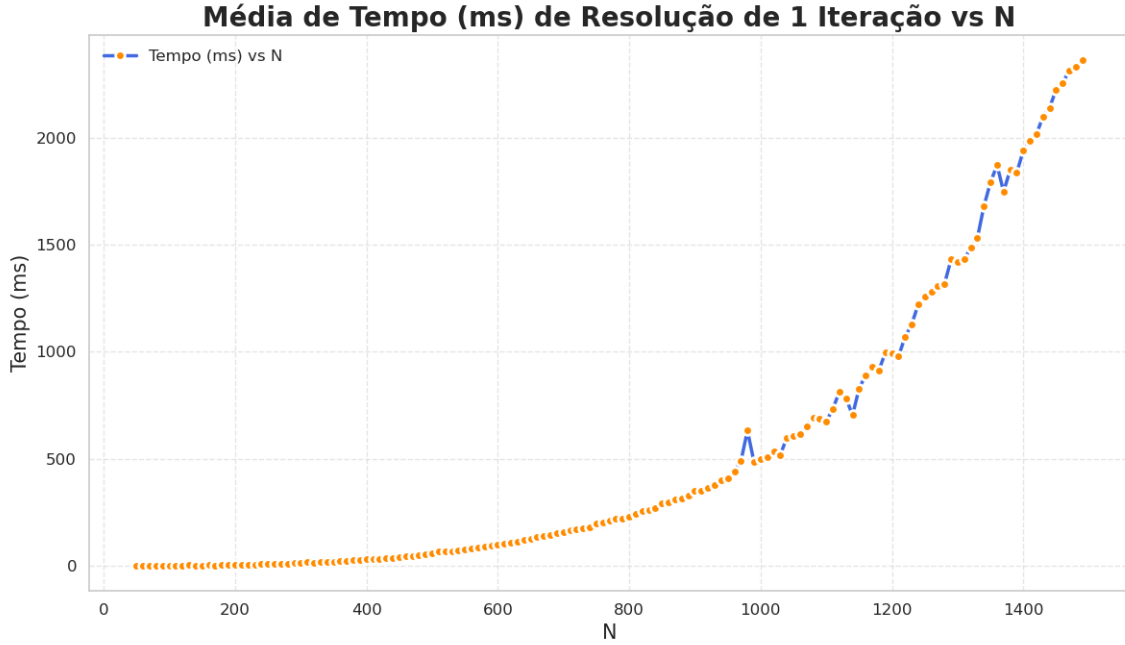


Figura 2: Tempo necessário para calcular uma iteração para uma matriz de restrições $N \times N$. Elaborado pelos autores (2025).

4.2.2 Tempo de Execução para PLs de Tamanho Fixo

Como o tempo de resolução de um PL não está somente atrelado ao tempo de iteração, uma vez que diferentes PLs precisam de quantidades diferentes de iterações para serem resolvidos, faz-se necessário avaliar o tempo de resolução de diferentes PLs de um mesmo tamanho. Para isso, aleatorizamos 100000 PLs "comportados" e verificamos a distribuição da quantidade de tempo que o código toma para resolvê-los. Fixamos o valor do intervalo de distribuição de valores para cada elemento da matriz A como uma distribuição uniforme de intervalo $(0, 100000)$. A escolha para o tamanho de $N \times N$ foi de 100×100 , 250×250 e 500×500 .

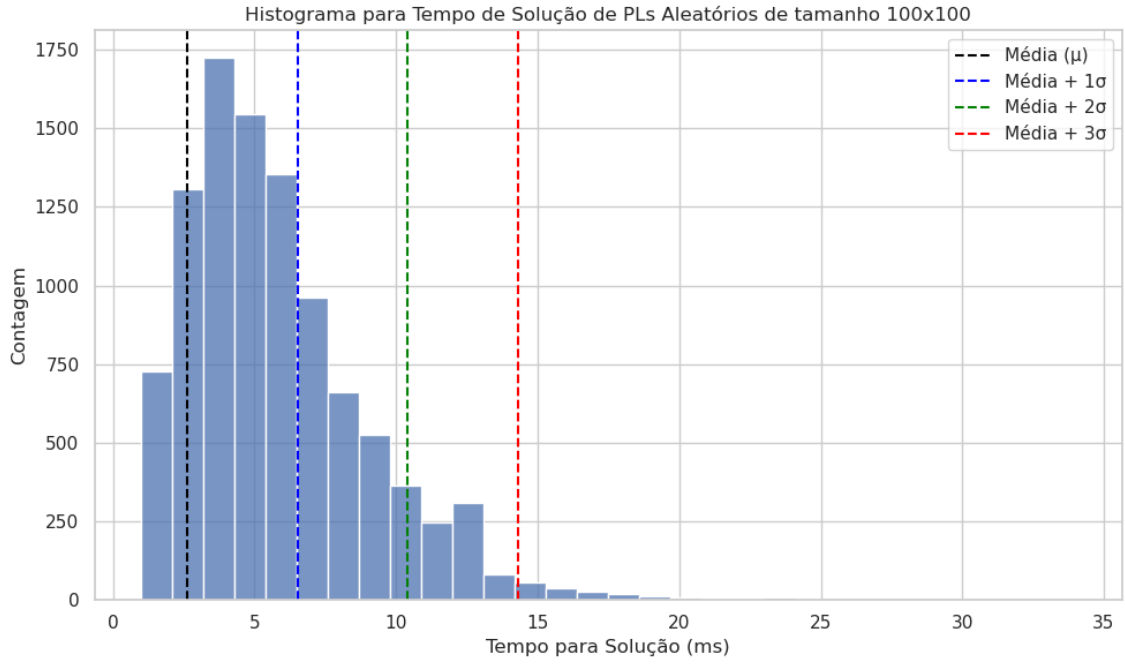


Figura 3: Histograma de tempo necessário para solucionar um PL para uma matriz de restrições 100×100 . Fonte: Elaborado pelos autores (2025)

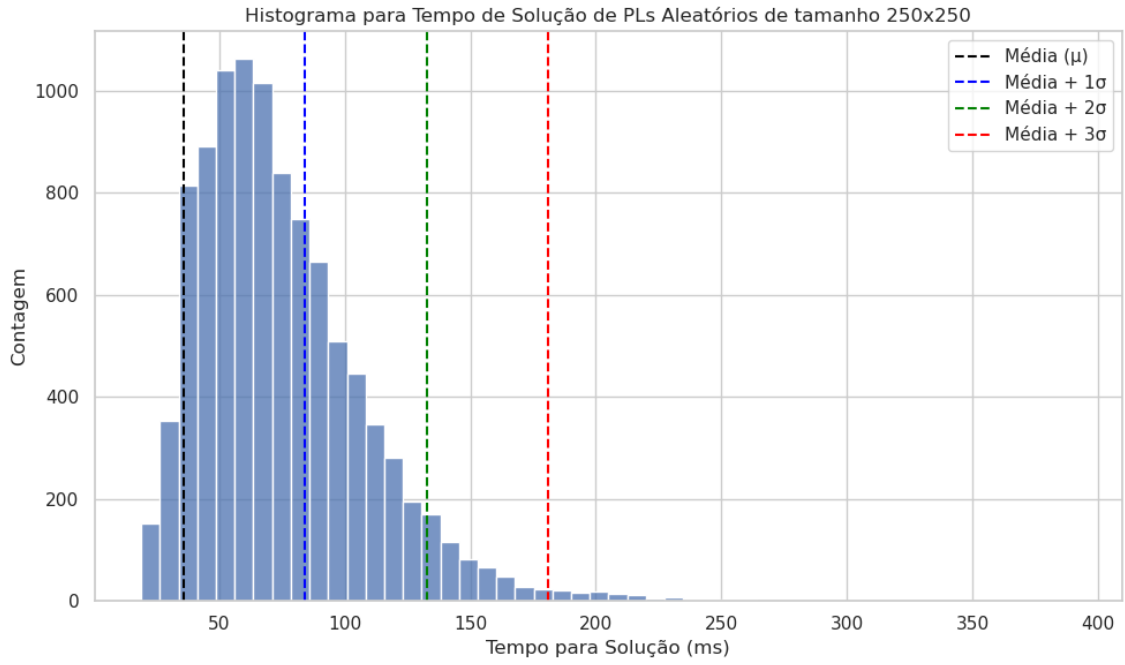


Figura 4: Histograma de tempo necessário para solucionar um PL para uma matriz de restrições 250×250 . Fonte: Elaborado pelos autores (2025)

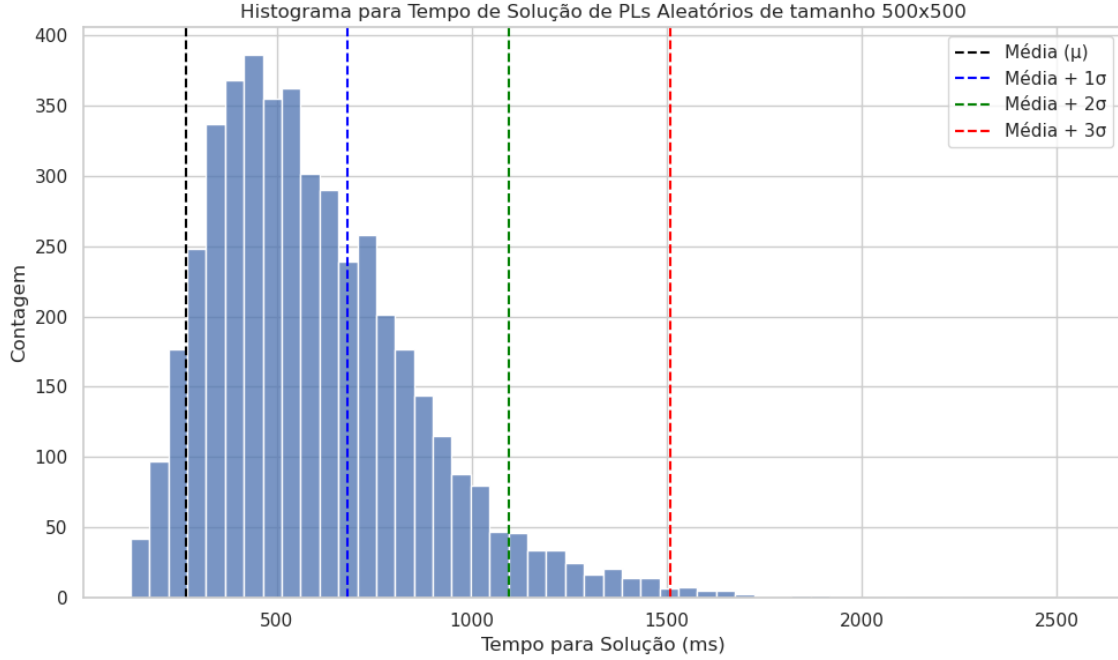


Figura 5: Histograma de tempo necessário para solucionar um PL para uma matriz de restrições 500×500 . Fonte: Elaborado pelos autores (2025)

4.2.3 Média de Tempo de Execução $\times$ Resolução de PLs

A fim de avaliar o tempo médio de resolução de Programas Lineares (PLs) de diferentes tamanhos, foi realizada uma análise do tempo necessário para resolver múltiplas instâncias de PLs de tamanho $N \times N$. Para isso, foram gerados e resolvidos 10 PLs aleatórios para cada tamanho $N \times N$, com N variando de 10 a 1100 em incrementos de 10. O tempo médio de resolução foi então calculado para cada grupo de instâncias, permitindo uma comparação da eficiência do algoritmo conforme o aumento da dimensão da matriz de restrições.

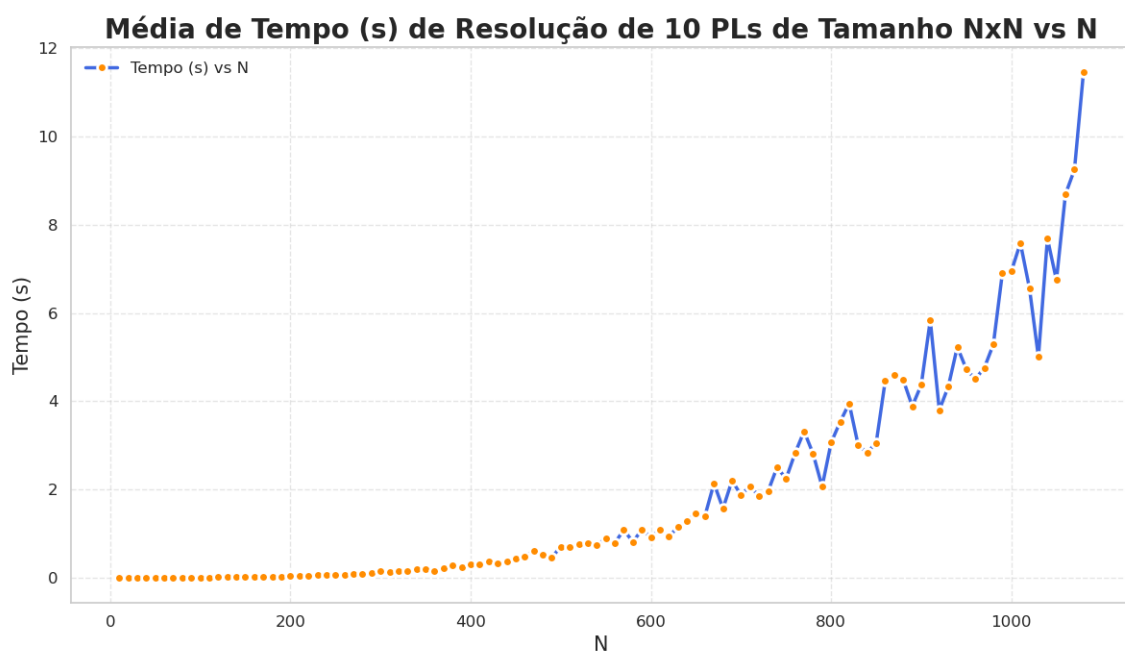


Figura 6: Linemap para média de tempo de resolução de 10 PLs de tamanho $N \times N$. Elaborado pelos autores (2025).

5 Discussão

O método implementado obteve resultados positivos na resolução de Problemas Lineares, uma vez que os nossos testes obtiveram velocidade satisfatória para problemas médios e pequenos. Particularmente, podemos utilizar as Figura 2 e 6 como base para uma análise mais minuciosa da implementação.

A Figura 2 demonstra que uma iteração do algoritmo performa condizentemente com o obtido por uma análise de complexidade assintótica de uma iteração - $O(n^3)$. Isso se deve, principalmente, à resolução dos sistemas lineares pelo método de eliminação gaussiana. Geralmente, utiliza-se fatoração LU para resolução de sistemas lineares para *Solvers* de PLs - uma abordagem $O(n^2)$. O algoritmo implementado, visando simplicidade, utiliza um algoritmo de pior performance à custo de complexidade de escrita e manutenção de código.

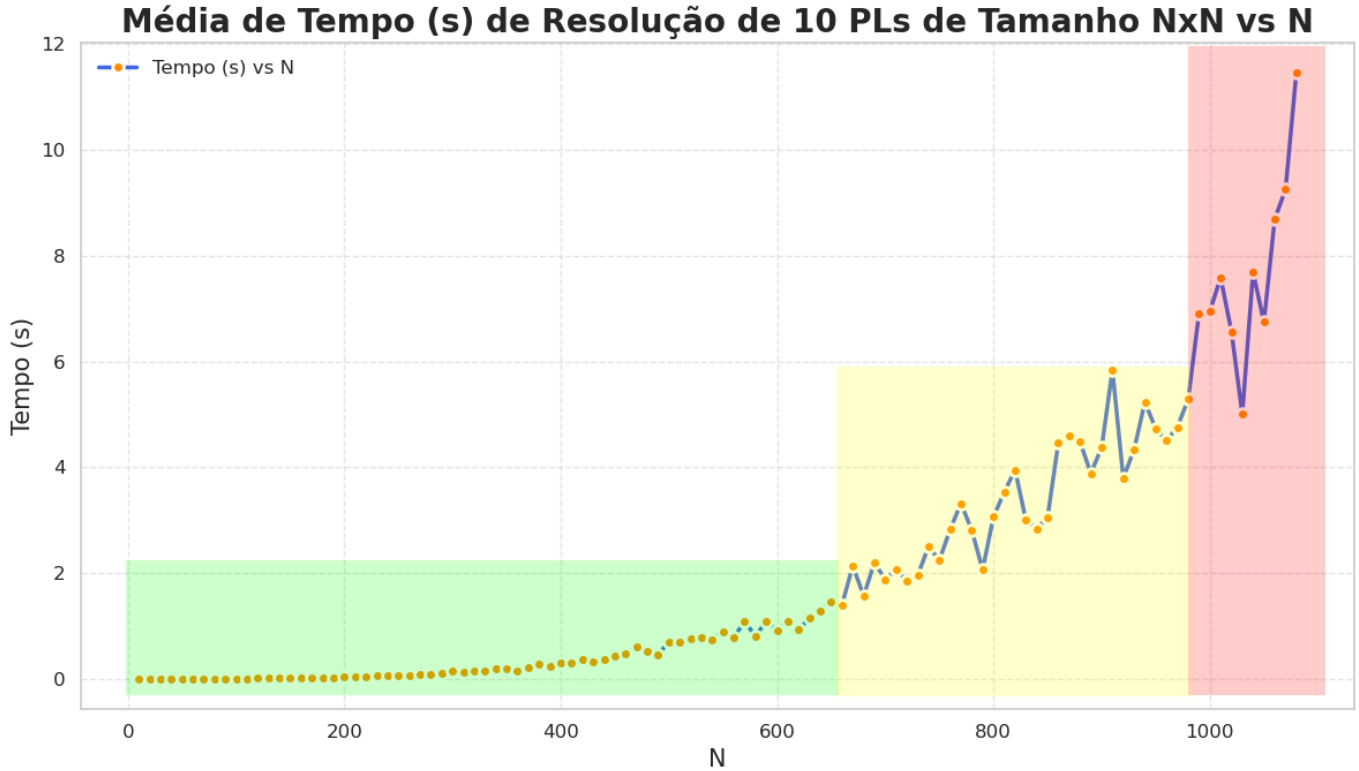


Figura 7: Linemap para média de tempo de resolução de 10 PLs de tamanho $N \times N$, com destaque especial a áreas de interesse. Elaborado pelos autores (2025).

Outra análise de interesse advém da Figura 7. Conseguimos identificar, no gráfico, três áreas de interesse:

1. A primeira área, destacada em verde, representa uma região do gráfico onde o tempo de execução permanece relativamente estável. Nessa área, a resolução do PL não apresenta variações significativas entre as instâncias, e os problemas são resolvidos de forma quase instantânea. Nesta área estão contidos os problemas de tamanho considerado pequeno.
2. A segunda área, destacada em amarelo, indica uma região de aumento no tempo de execução. Nela, o tempo necessário para resolver os PLs não é excessivamente longo, mas apresenta uma certa instabilidade em relação ao número de iterações e ao tempo total de execução. Esta área abrange problemas de tamanho médio.

3. A última área, destacada em vermelho, marca uma região onde o tempo de resolução se torna computacionalmente significativo. Nessa região, a solução dos problemas de maior tamanho exige um tempo considerável e reflete a maior complexidade computacional associada a esses casos.

Para tanto, é válido dizer que o algoritmo implementado cumpre com os objetivos definidos - os problemas de tamanho $< 1000 \times 1000$ são resolvidos em tempo satisfatoriamente rápido.

Por último, as figuras 3, 4 e 5 são de suma importância para validação do projeto. Espera-se que PLs gerados como uma distribuição uniforme também sejam representativamente resolvidos como distribuição uniforme. Os resultados, portanto, possuem distribuição condizente com o dataset.

6 Conclusão

O objetivo principal deste projeto é resolver problemas de programação linear de tamanho médio e pequeno. Nesse contexto, os resultados obtidos até o momento estão em conformidade com as metas estabelecidas, uma vez que o foco está em otimizar a solução de instâncias que não exigem um tempo computacional excessivo. Assim, o trabalho busca atender a demandas práticas, garantindo eficiência e viabilidade para a resolução de problemas de menor escala, que são típicos em diversas aplicações do mundo real.

Além disso, durante o desenvolvimento, identificamos algumas melhorias que poderiam aumentar a eficiência do código. O uso de matrizes esparsas, por exemplo, poderia reduzir o custo de memória para o armazenamento de matrizes. Embora mais eficientes para representar Programas Lineares (PLs) devido à grande quantidade de zeros, matrizes esparsas utilizam 12 bits por caractere não-zero, em comparação com os 8 bits por dígito utilizados na representação convencional. Para otimizar esse processo, seria interessante adotar uma heurística para identificar e converter automaticamente as matrizes para a forma esparsa durante a execução, reduzindo assim o consumo de memória e melhorando a performance. Além disso, percebemos que a escolha do método de eliminação de Gauss para a resolução dos sistemas lineares não foi a mais eficiente. A utilização da decomposição LU poderia ser uma solução mais adequada, pois permite uma abordagem mais eficiente para resolver sistemas lineares, especialmente em matrizes grandes ou mal condicionadas.

Referências

- [1] ARENALES, M.; ARMENTANO, V.; MORABITO, R.; YANASSE, H., *Pesquisa Operacional*, Ed. Campus, 2006.
- [2] George B. Dantzig y Mukund N. Thapa, *Linear Programming 1: Introduction*, Springer-Verlag, 1997.
- [3] G. B. Dantzig, A. Orden, and P. Wolfe, *The generalized simplex method for minimizing a linear form under linear inequality restraints*, Pacific Journal of Mathematics, vol. 5, no. 2, pp. 183–195, 1955.
- [4] Katta G. Murty, *Linear Equations, Inequalities, Linear Programs (LP), and a New Efficient Algorithm*, Department of Industrial and Operations Engineering, University of Michigan, Ann Arbor, MI 48109-2117, USA, 2006.
- [5] Kurgalin, Sergei; Borzunov, Sergei (2021). *Algebra and Geometry with Python*. Cham. doi: 10.1007/978-3-030-61541-3. ISBN 978-3-030-61540-6.
- [6] Mykhailova, Mariia. (2023). *Gaussian Elimination for Solving Linear Systems*. Disponível em: https://cp-algorithms.com/linear_algebra/linear-system-gauss.html.
- [7] Peter Sanders. *Algorithm Engineering*. In: David Padua, editor. *Encyclopedia of Parallel Computing*. Springer, 2011. p. 33–38.

- [8] Cai, S. W. (n.d.). *DIMACS Graph Format*. Recuperado de http://lcs.ios.ac.cn/~caisw/Resource/about_DIMACS_graph_format.txt. Shaowei Cai, Institute of Software, Chinese Academy of Sciences, Beijing, China.

A Prints - Testes de factibilidade

```
-----  
Min of  
-1x0 + -1x1 + -1x2 + 0x3 + 0x4  
/  
Subjected to:  
  
1 0 -1 1 0 = 2  
0 1 -1 0 1 = 3  
  
---  
---  
Iteration 0  
---  
  
Enters Non basic column: 2  
Lowest Relative Cost: -3  
  
f(x) = -5  
Problem is unbounded (no positive y).  
The problem is unbounded.  
Simplex Method - Time elapsed: [2ms]
```

Figura 8: Problema Ilimitado. Elaborado pelos autores (2025).

```

Min of
1x0 + 1x1

Subjected to:

1 1 = 3
1 1 = 2

---
---
Iteration 0
---

Enters Non basic column: 0
Lowest Relative Cost: -2e+09

f(x) = 5e+09
Basic column that leaves basis: 3
---
Iteration 1
---

Enters Non basic column: 1
Lowest Relative Cost: 0
Exited. Optimal solution found.

f(x) = 1e+09
INFEASIBLE: artificial variable x[2] = 1 > 1e-07
Simplex Method - Time elapsed: [0ms]

```

Figura 9: Problema Infactível. Elaborado pelos autores (2025).


```
Min of
-22x0 + -6x1 + -2x2 + 0x3 + 0x4 + 0x5
```

```
Subjected to:
```

```
10 2 1 1 0 0 = 100
7 3 2 0 1 0 = 72
2 4 1 0 0 1 = 80
```

```
---
```

```
---
```

```
Iteration 0
```

```
---
```

```
Enters Non basic column: 0
```

```
Lowest Relative Cost: -22
```

```
f(x) = 0
```

```
Basic column that leaves basis: 3
```

```
---
```

```
Iteration 1
```

```
---
```

```
Enters Non basic column: 1
```

```
Lowest Relative Cost: -1.6
```

```
f(x) = -220
```

```
Basic column that leaves basis: 4
```

Figura 10: Parte 1 - Problema Factível. Elaborado pelos autores (2025).

```
---  
Iteration 2  
---  
  
Enters Non basic column: 4  
Lowest Relative Cost: 1  
Exited. Optimal solution found.  
  
f(x) = -222  
  
-----  
  
Final solution:  
  
f(x) = -222  
  
x[0] = 9.75  
x[1] = 1.25  
x[2] = 0  
x[3] = 0  
x[4] = 0  
x[5] = 55.5  
Simplex Method - Time elapsed: [0ms]
```

Figura 11: Parte 2 - Problema Factível. Elaborado pelos autores (2025).

B Código em Python para Geração de PLs

```
def generateLP(b_size, c_size, interval_begin, interval_end):

    total_vars = c_size + b_size

    with open(f"LP_{c_size}x{b_size}.out", "w+") as f:

        f.write(f"p {total_vars} {b_size}\n")

        for i in range(b_size):
            f.write("a ")

            for j in range(c_size):
                f.write(f"{random.randrange(interval_begin, interval_end)} ")

            for k in range(b_size):
                if k == i:
                    f.write("1 ")
                else:
                    f.write("0 ")
            f.write("\n")

        f.write("b ")
        for i in range(b_size):

            val = random.randrange(interval_begin, interval_end)
            f.write(f"{val} ")
        f.write("\n")

        f.write("c ")

        for i in range(c_size):
            val = random.randrange(interval_begin, interval_end)
            f.write(f"{-val} ")

        for i in range(b_size):
            f.write("0 ")
        f.write("\n")
```